

ASTRE - Lab Assignment 3

Amy van der Meijden (5301513) and Jimmy Oei (6540031)

May 2026

Task 2

Algorithm 1 Using Tarantula scores to guide operator patching

```
1: Initialize mutateOperatorCount  $\leftarrow$  15
2: results  $\leftarrow$  runAllTests(currentOperators)
3: executedOperators  $\leftarrow$  record for each failed and passed test which operator IDs are executed
4: for each op in currentOperators do
5:   Count failed(op): failing tests that executed op.
6:   Count passed(op): passing tests that executed op.
7:   failRate  $\leftarrow$  failed(op)/totalFailing
8:   passRate  $\leftarrow$  passed(op)/totalPassing
9:   tarantulaScore(op)  $\leftarrow$  failRate/(failRate + passRate)
10: end for
11: mutableOperators  $\leftarrow$  top mutateOperatorCount from executedOperators.sortby(tarantulaScore)
12: patch  $\leftarrow$  mutate only the mutableOperators
```

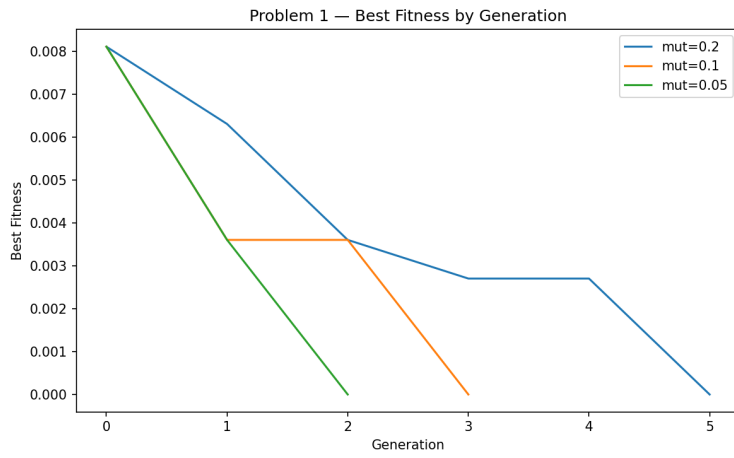
Task 3

We measured the fitness over 200 generations for all problems., with a mutation rate of 0.2, 0.1, and 0.05. Below, one can find the plot for each RERS problem. In some cases, the program stopped running, when the fitness value did not decrease any more. Crossover was not implemented.

Problem 1

Mutation Rate	Best Fitness
0.05	0.000000
0.10	0.000000
0.20	0.000000

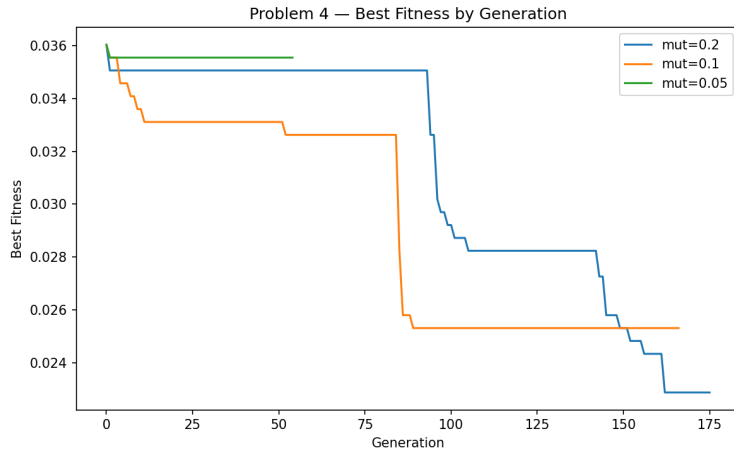
Table 1: Best fitness scores by mutation rate for Problem 1



Problem 4

Mutation Rate	Best Fitness
0.05	0.035540
0.10	0.025316
0.20	0.022882

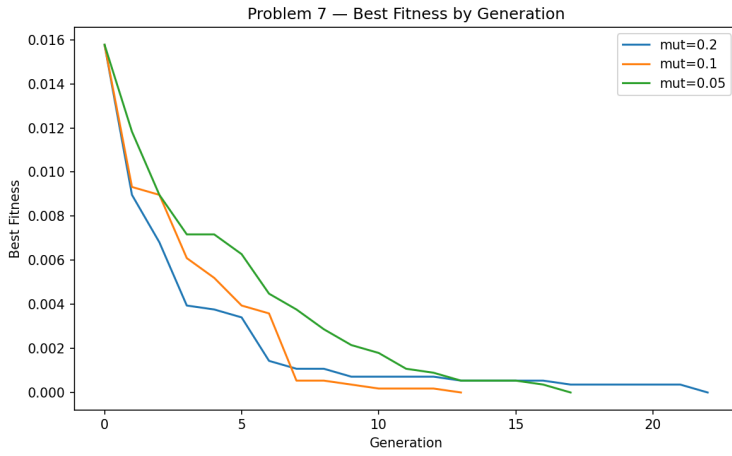
Table 2: Best fitness scores by mutation rate for Problem 4



Problem 7

Mutation Rate	Best Fitness
0.05	0.000000
0.10	0.000000
0.20	0.000000

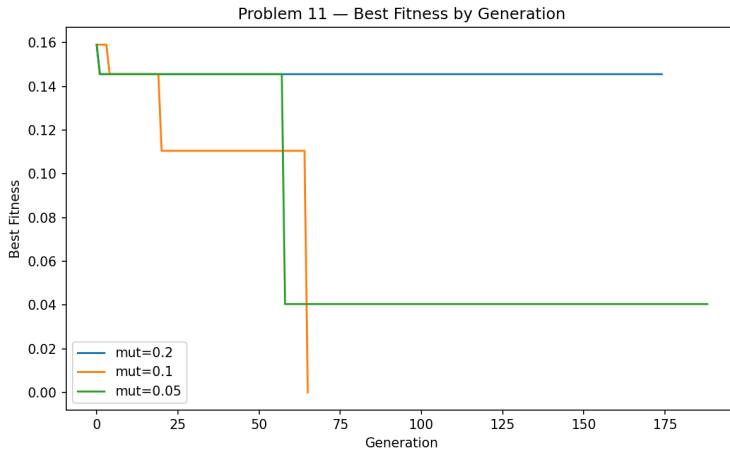
Table 3: Best fitness scores by mutation rate for Problem 7



Problem 11

Mutation Rate	Best Fitness
0.05	0.040431
0.10	0.000000
0.20	0.145553

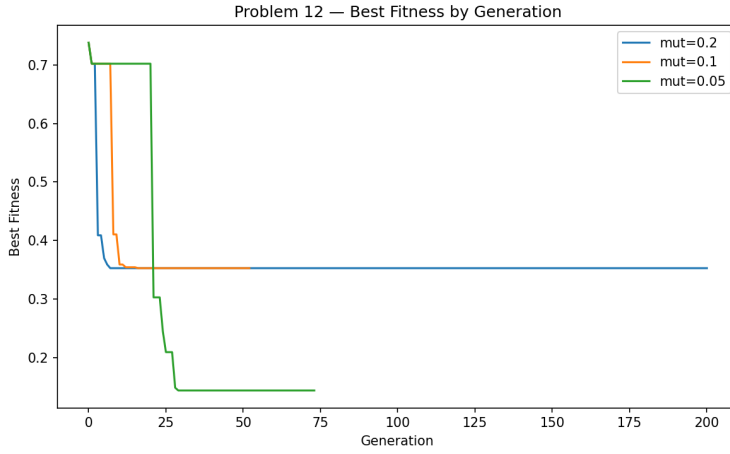
Table 4: Best fitness scores by mutation rate for Problem 11



Problem 12

Mutation Rate	Best Fitness
0.05	0.143526
0.10	0.352574
0.20	0.352574

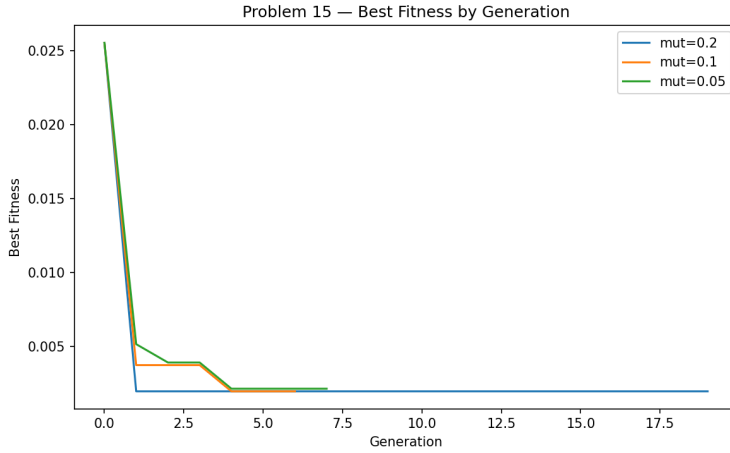
Table 5: Best fitness scores by mutation rate for Problem 12



Problem 15

Mutation Rate	Best Fitness
0.05	0.002129
0.10	0.001952
0.20	0.001952

Table 6: Best fitness scores by mutation rate for Problem 15



Correctly Patched Faults

To determine how many faults were correctly patched, we compared the patches generated by our EA against the operator lists from the instrumented original programs. A patch was counted as correct only if the patched operator matched the operator used in the original program.

Problem	Patch changes	Final fitness	Correctly patched faults
Problem 1	4	0.0000	0
Problem 4	2	0.0355	0
Problem 7	15	0.0000	0
Problem 11	6	0.0404	0
Problem 12	11	0.1435	0
Problem 15	4	0.0021	0

Table 7: Number of correctly patched faults per buggy RERS problem.

Although the EA improved the fitness for all problems and reached zero failing tests for Problems 1 and 7, none of the generated operator changes matched the original operators. Therefore, the number of correctly patched faults is 0 for all six buggy problems. This indicates that the EA found patches that passed the tests or improved test results, but they were not the expected operators according to the original program. The EA was unable to recover the original program.

Task 4

We ran ASTOR on these six buggy RERS problems: 1, 7, 11, 12, 13, and 14. The results can be seen in table 8.

Problem	Patch changes	ASTOR time (s)	Correctly patched faults
Problem 1	1	23.6	1
Problem 7	0	301.0	0
Problem 11	1	16.0	1
Problem 12	1	1.9	1
Problem 13	1	151.8	1
Problem 14	1	57.6	1

Table 8: ASTOR results per buggy RERS problem. *Patch changes* is the number of AST hunks in the validated patch (0 if none was found within the budget).

We found that ASTOR generates a correct patch for 5 out of 6 analyzed problems. Only problem 7 hit a time out and could not find a correct patch, this could be caused by its larger size versus the other 5 problems as it contains 10k lines of code whereas the other 5 problems are all less than 4k lines of code, which means problem 7 likely hit the 5 minutes time out before a correct patch could be found. ASTOR finds meaningful patches because they make edits per one operator instead of the whole operator vector. This leads to more targeted updates compared to our EA where we mutate about 20% of the operator vector.

References

- [1] Source code repository: <https://github.com/AmyAmyy/ASTRE26/tree/Lab3>